A 5-DAY COURSE IN MARKET STRUCTURE & EXCHANGE TECHNOLOGY

EduMatcher
Inside a Modern Equity Trading System

From a single order to a fully wired exchange — order books, auctions, market makers, risk controls and post-trade, built up one session at a time.

5 DAYS | 20 LECTURES | 5 HANDS-ON LABS | 1 LIVE EXCHANGE

Johan Persson, 2026

The week at a glance

DAY 01

FOUNDATIONS

Anatomy of an exchange, the matching engine, the order book & gateways

DAY 02

ORDER TYPES

From MARKET & LIMIT to stops, icebergs, OCO brackets & multi-leg combos

DAY 03

MARKET STRUCTURE

Session phases, opening & closing auctions, market making & automated liquidity

DAY 04

RISK & POST-TRADE

Price collars, circuit breakers, kill switch, P&L, clearing & compliance

DAY 05

DATA & OPERATIONS

Protocols, statistics, the market index, operating & running a full exchange

Each day builds on the last — by Friday every box below is a live, connected process.

How each teaching day is structured

1	2	3	4	5	6	7
OBJECTIVES	**LECTURE 1**	**LECTURE 2**	**LAB**	**LECTURE 3**	**LECTURE 4**	**RECAP**
Start of day	Morning	Morning	Midday	Afternoon	Afternoon	End of day
What you must be able to do by tonight	Concept introduced	Concept deepened	Hands-on exercise on the live engine	Apply & extend	Bring it together	Consolidate the day's ideas

Objectives first

Every session opens with a short list of exactly what you should know or be able to do afterwards.

Learn by doing

Two lectures, then a lab where you drive the real engine yourself, then two more lectures.

Recap to retain

Each day closes by revisiting its key ideas so they stick before the next layer is added.

Anatomy of an electronic exchange

Every exchange — from EduMatcher on your laptop to the NYSE — is built from the same functional layers. This week we light them up one by one.

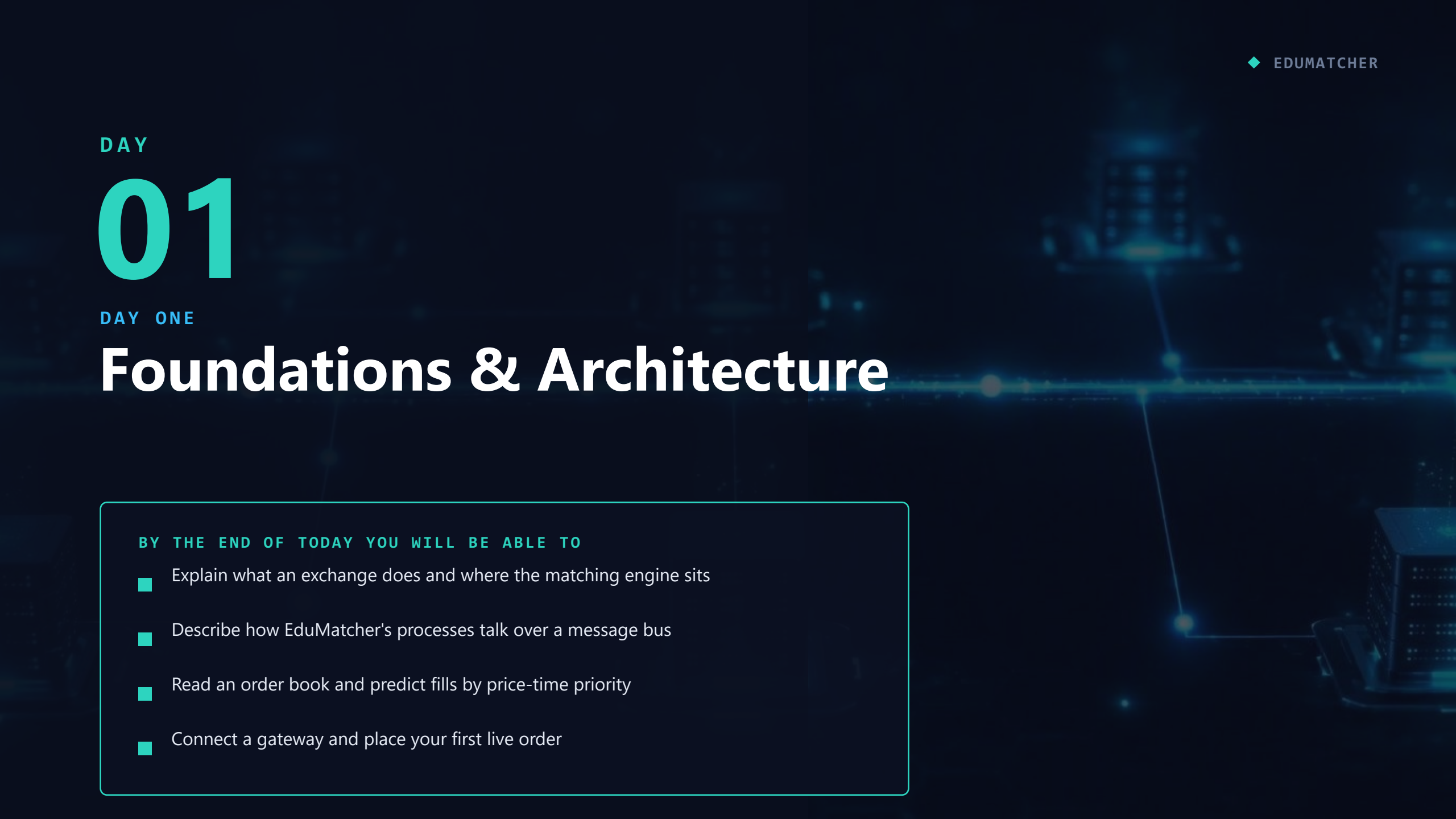
01	PARTICIPANTS	Traders, market makers & bots send orders in	Day 1 · 3
02	CONNECTIVITY	Gateways & protocols carry messages in and out	Day 1 · 5
03	MATCHING ENGINE	The order book that crosses buyers & sellers	Day 1 · 2
04	MARKET CONTROLS	Sessions, auctions, risk limits & circuit breakers	Day 3 · 4
05	POST-TRADE & DATA	Clearing, P&L, audit, statistics & the index	Day 4 · 5

DAY

01

DAY ONE

Foundations & Architecture



BY THE END OF TODAY YOU WILL BE ABLE TO

- Explain what an exchange does and where the matching engine sits
- Describe how EduMatcher's processes talk over a message bus
- Read an order book and predict fills by price-time priority
- Connect a gateway and place your first live order

What is an exchange — and what is EduMatcher?

The exchange is the neutral venue

- **Price discovery.** It brings buyers and sellers together and publishes the price they agree on.
- **It never takes a position.** The venue only matches participant orders — it is an impartial referee.
- **A matching engine at the core.** One authoritative process holds the book and crosses compatible orders.
- **Same mechanics as NYSE.** Continuous book, auctions, market makers, risk controls — just simplified.

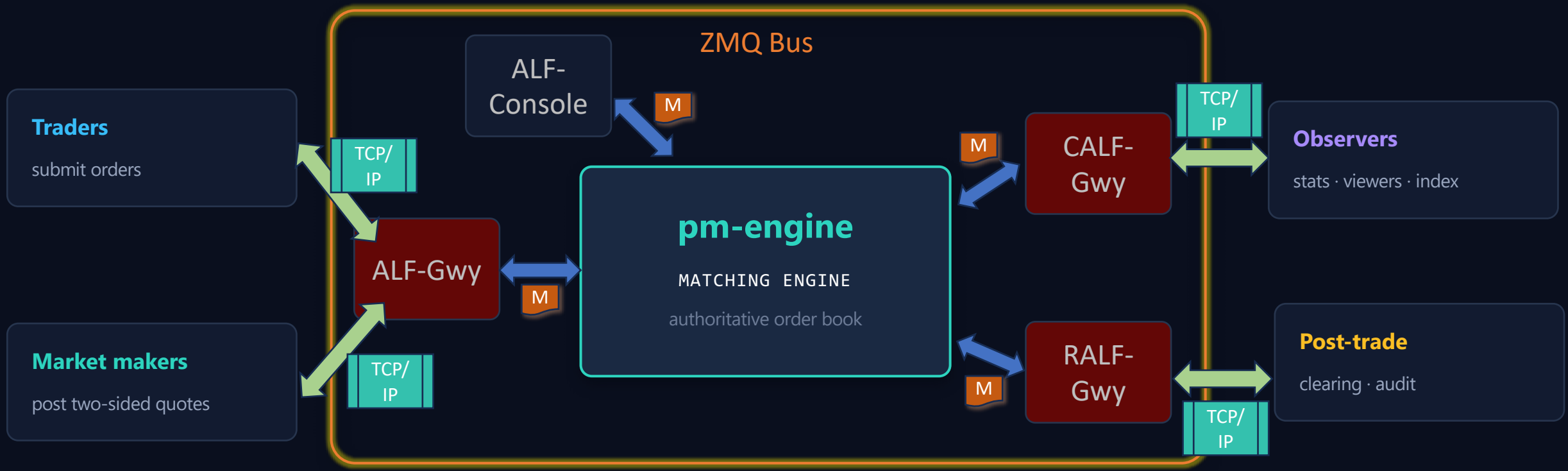
EDUMATCHER

A fully functional matching engine, built to teach.

- **Continuous order book** — matches buyers & sellers live
- **Opening & closing auctions** — single-price uncrossing
- **Market-maker quoting** — obligations & protection
- **Risk controls** — collars, breakers, kill switch
- **Full post-trade** — P&L, clearing, audit, drop-copy

The matching engine: one book, one writer

pm-engine is the single writer of the order book. Every other process only submits orders or observes events — this is what keeps matching deterministic and fair.



Orders flow IN over a PUSH socket · events flow OUT over a PUB socket — covered next.

The process constellation

EduMatcher is not a monolith. Around a dozen small pm-* processes each do one job and coordinate over a ZeroMQ message bus.

ORDER ENTRY

pm-alf-console
interactive trader terminal

pm-alf-gwy
TCP gateway for bots

pm-balf-gwy
binary low-latency entry

THE CORE

pm-engine
matching engine & book

pm-scheduler
drives session phases

pm-index
cap-weighted index

MONITOR & DATA

pm-stats
OHLCV / VWAP recorder

pm-clearing
positions & running P&L

pm-audit
full event logger

EXTERNAL FEEDS

pm-md-gwy
market data (CALF)

pm-ralf-gwy
post-trade (RALF)

pm-api-gwy
REST / WebSocket API

The message bus: one publisher, many listeners

The core engine never calls anyone. It publishes every event onto a ZeroMQ bus, and any process that cares simply subscribes and reads.

PUBLISH, DON'T CALL

The engine fires each event onto the bus and moves on. It never blocks, waits for a reply, or knows who is listening.

SUBSCRIBE BY TOPIC

Each process opens a SUB socket and filters by topic prefix, so it receives only the messages it actually cares about.

ONE-TO-MANY, DECOUPLED

Add or drop listeners freely. The same event reaches every subscriber and the engine's code never has to change.

The message bus: three sockets, two patterns

:5555

PULL

COMMANDS IN

Clients PUSH orders, cancels & admin here. One-to-one, reliable delivery.

:5556

PUB

EVENTS OUT

Acks, fills, trade.executed, book & session updates. One-to-many, topic-filtered.

:5557

PUB

DROP-COPY

A parallel per-fill feed with sequence numbers for risk & compliance systems.

Why two patterns?

- **PUSH / PULL** — reliable one-to-one delivery, used for commands the engine must not miss.
- **PUB / SUB** — broadcast, best-effort; a late subscriber misses earlier messages.

Startup order matters. The engine binds the sockets, so it must start first; everything else connects to it. That is why the engine also publishes a book snapshot on startup — so late subscribers can catch up.

Lab: Your five-minute exchange

EXERCISE

Your goal

Bring up a minimal exchange using a Multipass VM, connect two traders, and cross a single trade — proving you understand engine vs. participant roles.

YOU SHOULD SEE
A FILL at 100 @ 150.00 on both terminals and one trade.executed event.

Step by step

0 **Open Three Multipass Shell**
`pmultipass shell edumatcher-<version>`

1 **Start the engine on shell 1**
`cd session`
`pm-engine --verbose` binds ports 5555 / 5556

2 **Open two ALF-consoles as** `TRADER01` (buyer) and `TRADER02` (seller)
`pm-alf-console --id TRADER01`
`pm-alf-console --id TRADER02`

3 **Seller sells to a MM with a LIMIT order:**
[TRADER02]> `NEW|SIDE=SELL|SYM=AAPL|TYPE=LIMIT|QTY=100|PRICE=150.00`

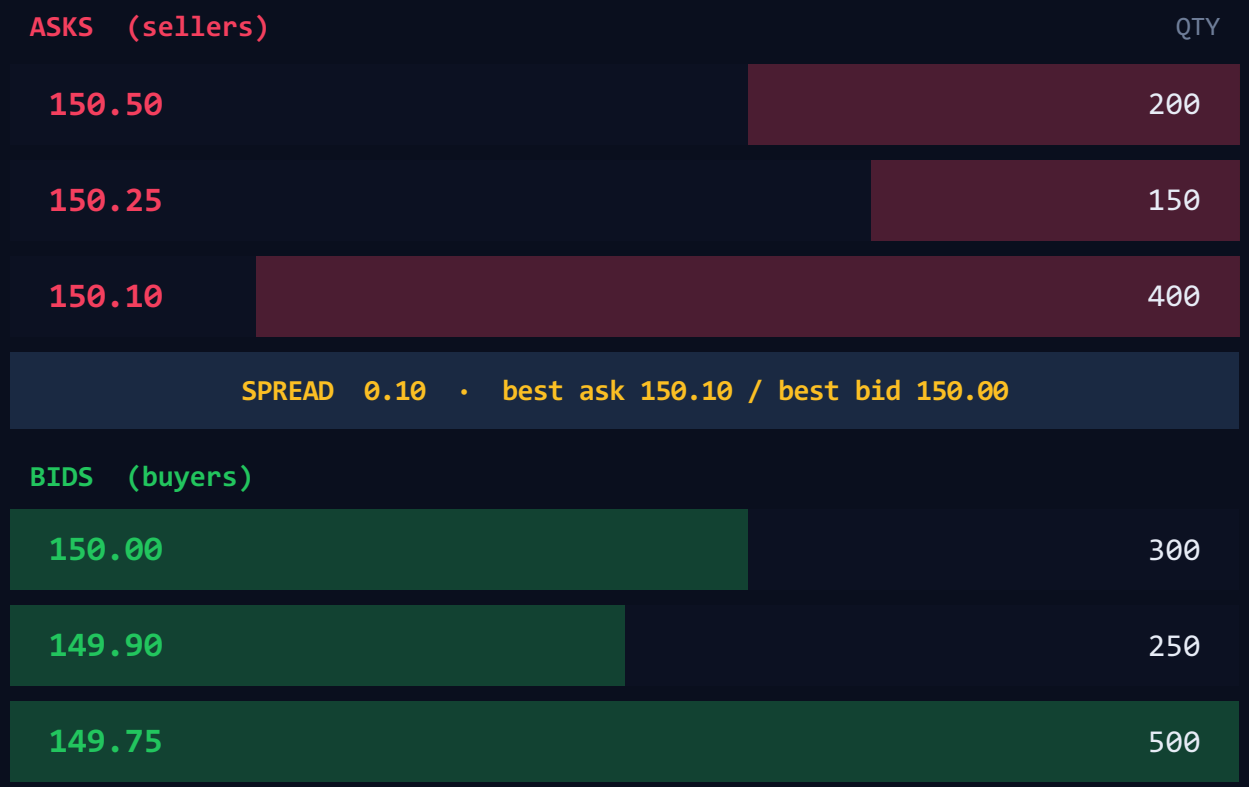
4 **Seller rests a LIMIT order:**
[TRADER02]> `NEW|SIDE=SELL|SYM=AAPL|TYPE=LIMIT|QTY=100|PRICE=250.00`

5 **Buyer buys from MM with LIMIT order:**
[TRADER01]> `NEW|SIDE=BUY|SYM=AAPL|TYPE=LIMIT|QTY=100|PRICE=250.00`

6 **Say out loud** what the engine did vs. what each participant did. What price did TRADER01 pay for the order? (Why?)

The order book & price-time priority

The book is a sorted list of resting bids and asks. The rule: better prices go first; among equal prices, the earlier order goes first.



1 Best bid & best ask

Highest buy price and lowest sell price sit at the top of the book — the touch.

2 Time breaks ties

Two orders at the same price? The one that arrived first is filled first.

3 Rest or cross

If a new order's price meets the other side it crosses now; otherwise it rests.

How a fill happens — a worked cross

1

GW02 rests a sell

SELL 100 @ 150.00 has no buyer yet, so it rests as the best ask.

2

GW01 sends a buy

BUY 100 @ 150.00 — the bid price now meets the ask price.

3

The engine crosses

Bid ≥ ask, so the two orders match instantly at 150.00.

4

Events fan out

order.fill to each side + one global trade.executed on the bus.

```
GW02> NEW|SYM=AAPL|SIDE=SELL|TYPE=LIMIT|QTY=100|PRICE=150.00
      [09:30:00.10] ACK  order resting
GW01> NEW|SYM=AAPL|SIDE=BUY|TYPE=LIMIT|QTY=100|PRICE=150.00
      [09:30:01.42] FILL 100 @ 150.00  [FILLED]   → trade.executed
```

Connecting as a participant: the ALF command

ALF — “Almost FIX”

A pipe-delimited KEY=VALUE text format, inspired by FIX but stripped of its session layer, checksums and sequence numbers — readable enough to type by hand.

- **NEW** message type
- **SYM=AAPL** instrument
- **SIDE=BUY** direction
- **TYPE=LIMIT** order type
- **QTY=100** size
- **PRICE=150.00** limit price

Identity is everything

Your gateway --id is the only account the engine knows. Every order from GW01 is one book of orders and one P&L.

TRADER MARKET_MAKER ADMIN

```
TRADER01> NEW|SYM=AAPL|SIDE=BUY|TYPE=LIMIT|QTY=100|PRICE=150.00
[09:30:00.10] ACK 7c4a91e2  order accepted
TRADER01> AMEND|ID=7c4a91e2|PRICE=150.10
[09:30:01.50] AMENDED price=150.10 qty=100
TRADER01> EXIT
```

The gateway session lifecycle

Every trading session follows the same four beats — from the first handshake to a clean exit. The engine authenticates once, then keeps the slot open for the whole conversation.

1

CONNECT

Gateway sends gateway_connect and waits up to 3 seconds for authentication.

2

AUTHENTICATE

Engine checks the id against the allowlist and replies gateway_auth.

3

TRADE

Interactive prompt: send NEW / AMEND / CANCEL, receive ack, fill & cancel events.

4

DISCONNECT

EXIT closes the session cleanly; the engine releases the gateway slot.

Three outcomes at auth: id on the allowlist → prompt opens · id unknown → rejected with a reason · no reply → "authentication timed out" (engine down).

Day 1 in review — foundations

1. **The exchange matches, it doesn't trade**
   A neutral venue for price discovery; pm-engine owns the book.
2. **A constellation of processes**
   ~12 pm-* processes cooperate — nothing is a monolith.
3. **Three sockets, two patterns**
   5555 in, 5556 out, 5557 drop-copy; PUSH/PULL + PUB/SUB.
4. **Price-time priority**
   Better price first; equal price, earlier order first.
5. **A cross makes a fill**
   Bid meets ask → match → fills + trade.executed.
6. **ALF over a gateway**
   Type KEY=VALUE orders; your --id is your account.

KEY TAKEAWAY An exchange is a message bus with a matching engine at its heart — everything else plugs into it.

DAY

02

DAY TWO

The Language of Orders

BY THE END OF TODAY YOU WILL BE ABLE TO

- Choose between MARKET and LIMIT and predict how each rests or fills
- Use STOP and STOP_LIMIT orders and trace their trigger logic
- Apply ICEBERG, FOK, IOC and TRAILING_STOP with the right Time-in-Force
- Manage orders with AMEND, and build OCO brackets & multi-leg combos

The order lifecycle & the two foundations

Every order walks the same path

NEW ▸ PARTIAL ▸ FILLED

…or ends early as CANCELLED · EXPIRED · REJECTED

MARKET Execution now, price uncertain

- **Sweeps the book** from the best price outward until filled.
- **Never rests** — any unfilled remainder is discarded.
- **Thin books → slippage** you pay through several price levels.

LIMIT Price guaranteed, fill uncertain

- **Price or better** — buy at ≤ your price, sell at ≥ your price.
- **Rests if unmatched** and becomes visible passive liquidity.
- **Builds the book** — the bid/ask ladder everyone prices against.

Slippage: the cost of demanding immediacy

A MARKET BUY for 100 shares sweeps the ask ladder. When the top level is thin, later shares fill higher — the average price drifts away from the touch.

ASK LADDER — swept bottom-up

150.00

Expected price

the touch you saw

150.04

Average fill

across three levels

The takeaway

- **MARKET** buys certainty of execution — and pays for it in price.
- **LIMIT** caps your price but may leave you unfilled.
- **Rule of thumb:** the thinner the book, the more MARKET costs.

Stops: dormant orders that wake on a trade

Stops watch the last executed trade price. When it reaches the stop, the order wakes up, converts, gets a fresh timestamp and re-enters matching.

Trigger logic

BUY STOP fires when
last ≥ stop (breakout / cover)

SELL STOP fires when
last ≤ stop (classic stop-loss)

STOP → MARKET on trigger

Fill certainty over price. Once the last trade crosses your stop, it becomes a market order and takes whatever is there — slippage possible.

```
SELL STOP
QTY=100
STOP=148.00
last 149.5 → 147.2
✓ fires → MARKET
```

STOP_LIMIT → LIMIT on trigger

Price control over certainty. On trigger it rests as a limit at your PRICE — protecting you, but it may sit unfilled if the market runs past.

```
SELL STOP_LIMIT
STOP=148.00
PRICE=147.50
triggers, market drops
↳ rests @ 147.50
```

Lab: Stop cascades & order-type bake-off

EXERCISE

Your goal

Feel how order types behave differently against the same book — and watch one aggressive print set off a chain of stops.

YOU SHOULD SEE
A cascade of three stop fills, then four visibly different outcomes.

Step by step

1. **Rest three SELL STOPs at** `149, 148, 147` on one symbol

2. **Push a trade print down to 147.00 and watch all three** fire in sequence (a cascade).

3. **Now send the same 100-share aggressive BUY four ways:** `LIMIT` · `IOC` · `FOK` · `MARKET`

4. **Tabulate each outcome:** rested vs. filled vs. rejected vs. remainder discarded.

Advanced execution orders

ICEBERG

Shows a small VISIBLE peak, hides the rest; refills with a new timestamp so it re-queues.

FOK

Fill-or-Kill: the whole quantity fills immediately or the entire order is cancelled.

IOC

Immediate-or-Cancel: take what's available now (partials ok), discard the rest.

TRAILING_STOP

A stop whose trigger ratchets in your favour by a fixed offset — never against you.

TRAILING_STOP in motion

SELL, trail 2.00. Price 100 → stop 98 · rallies to 110 → stop ratchets up to 108 · pulls back to 102 ≤ 108 → **TRIGGERED**. The floor moved up with the rally but never came back down.

Time-in-Force: how long an order lives

DAY

DEFAULT

Valid for this session only; expires at the close. The default.

GTC

PERSISTENT

Good-Till-Cancelled — persisted to disk and reloaded; survives across days.

ATO

AUCTION-ONLY

At-The-Open — accepted only during the opening auction.

ATC

AUCTION-ONLY

At-The-Close — accepted only during the closing auction.

⚠ Two classic mistakes

`TIF=FOK is invalid` — FOK is an order TYPE, not a time-in-force.

`IOC is rejected inside auctions` — it can't rest, and auctions don't match live.

AMEND: change an order without losing your place — sometimes

Amending a resting order keeps its ID, but whether it keeps its place in the queue depends on what you change.

PRIORITY PRESERVED

- **Quantity decrease only.** You keep your timestamp and stay where you are in line.

Good when you simply want less size on the book.

Also remember

- **QTY is the new total,** not an increment — and must exceed already-filled quantity.
- **Only LIMIT & ICEBERG** are amendable; change side or type via cancel + new.

PRIORITY LOST

- **Any price change,** or any quantity increase, sends you to the back of the queue.

```
BUY LIMIT 100 @ 150.00   (id abc123)
AMEND|ID=abc123|PRICE=151.00
  ↳ repriced, back of queue
AMEND|ID=abc123|QTY=80
  ↳ smaller, priority kept
```

Linked & multi-leg: OCO and COMBO

OCO — One-Cancels-Other

Two linked orders; when one reaches a terminal state, the engine cancels its sibling. The classic bracket around a position.

Long 100 AAPL @ 150

TAKE-PROFIT
SELL LIMIT @ 160

STOP-LOSS
SELL STOP @ 140

Rally to 162 → the limit fills, the stop auto-cancels. You can never be filled on both.

A partial fill does NOT cancel the sibling — only a terminal state does.

COMBO — Multi-leg, all-or-none

Bundle 2–10 legs across symbols as one unit. It only completes when every leg fully fills — solving "leg risk."

Pairs trade — profit from the spread, not direction

LEG 0	BUY 100 MSFT @ 415
LEG 1	SELL 100 AAPL @ 212

Cascade-cancel: if one leg is pulled, remaining unfilled legs cancel — but executed fills are never unwound.

Day 2 in review — order types

1. **MARKET vs LIMIT**
   Certainty of fill vs. certainty of price — the core trade-off.

2. **Stops wake on the last trade**
   Convert to market or limit, then re-enter with a new timestamp.

3. **FOK vs IOC**
   All-or-nothing vs. take-what's-there; both never rest.

4. **ICEBERG hides size**
   Only the peak shows; refills go to the back of the queue.

5. **AMEND & priority**
   Only a quantity decrease keeps your place in line.

6. **OCO & COMBO**
   Brackets that self-cancel; multi-leg strategies as one unit.

KEY TAKEAWAY Order types are a vocabulary for expressing intent — speed, discretion, protection or strategy.

DAY

03

DAY THREE

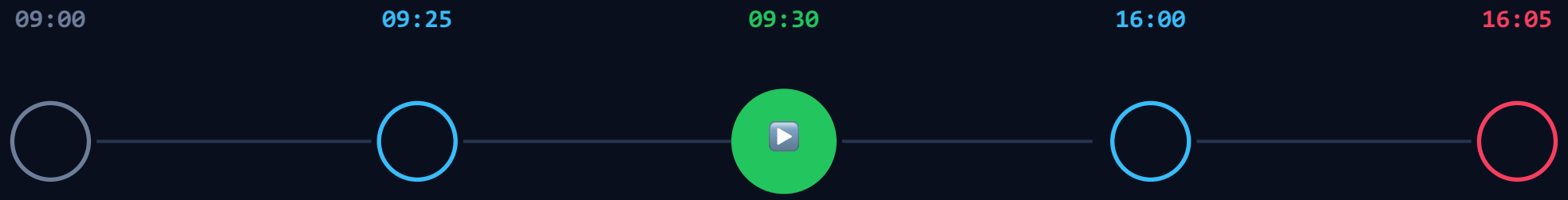
Sessions, Auctions & Liquidity

BY THE END OF TODAY YOU WILL BE ABLE TO

- Walk the five phases of the trading day and what each allows
- Explain why auctions exist and compute an equilibrium price
- Post a two-sided quote and describe market-maker obligations
- Use bots and AI traders to fill a classroom book with liquidity

The trading day has five phases

A trading day is not one continuous period. pm-scheduler moves the engine through phases at set times — and only one of them actually matches orders.



Only CONTINUOUS matches. In the auction and pre-open phases orders are accepted but rest. Every transition into a matching phase runs an uncross across every book.

What's allowed when: phase gating

	PRE_OPEN	OPEN AUCT.	CONTINUOUS	CLOSE AUCT.	CLOSED
LIMIT / ICEBERG	✓	✓	✓	✓	—
MARKET / FOK / IOC	—	—	✓	—	—
ATO orders	—	✓	—	—	—
ATC orders	—	—	—	✓	—
Matching happens	—	—	✓	—	—

MARKET, FOK and IOC need immediate execution, so they're rejected outside CONTINUOUS. ATO / ATC only live inside their matching auction.

Auctions: gather, then cross at one price

An auction collects orders over a window without matching, then computes a single equilibrium price and fills all crossable orders there at once — the uncross.

1

Opening price discovery

Everyone expresses interest at once after overnight news — no first-mover race.

2

A trusted closing price

The official close feeds NAVs, index rebalancing and settlement.

3

Less information asymmetry

Simultaneous execution neutralises speed advantages over stale orders.

4

Maximum shares traded

The algorithm explicitly picks the price that trades the most volume.

Finding the equilibrium price

Build cumulative demand and supply, then pick the price that executes the most shares. Ties break to the smaller surplus, then the lower price.

PRICE	CUM. BUY	CUM. SELL	EXECUTABLE
105.00	10	—	0
104.00	—	25	—
103.00	20	—	15 (surplus 5)
102.00	—	15	15 (surplus 5)
100.00	30	—	—

RESULT

102.00

equilibrium price

- **15 shares** execute — the maximum possible.
- **Tie 102 vs 103?** equal surplus → lower price wins.
- **Surplus of 5** on the buy side rests for the next phase.

All fills print at 102.00 — one price, no slippage, no improvement.

Lab: Drive an opening auction by hand

EXERCISE

Your goal

Take manual control of the session, load an auction with orders, and trigger the uncross to see the equilibrium price appear.

YOU SHOULD SEE
auction.result AAPL price=149.50 qty=100 surplus=0.

Step by step

1. **As ADMIN, open the auction:** `SESSION|STATE=OPENING_AUCTION`
2. **Enter two ATO orders — a** `BUY @ 150.00` and a SELL @ 149.50
3. **Uncross by advancing:** `SESSION|STATE=CONTINUOUS`
4. **Read the result event and ask:** why 149.50, not the midpoint?

Market makers: always two-sided

What a market maker does

- **Posts a bid and an ask** continuously, so a buyer always finds a seller and vice-versa.
- **Provides liquidity** — the market stays tradeable even when natural flow is thin.
- **Earns the spread** as compensation for standing ready and holding inventory.
- **Requires the role.** A plain TRADER sending QUOTE is rejected.

The QUOTE command

One atomic command posts both legs under a shared quote id.

```
QUOTE|SYM=AAPL|BID=149.90|ASK=150.10
     |BID_QTY=500|ASK_QTY=500
```

BID 149.90

buy up to 500

ASK 150.10

sell up to 500

`Refresh policy` decides what happens to the other leg when one side fills — cancel on any fill, on a full fill, or never.

Obligations keep quotes honest

When obligations are enforced, the engine validates every quote before it rests — a market maker must show a tight enough, large enough market.

Spread constraint

`(ask − bid) / tick ≤ max_spread_ticks`

Too wide a market is rejected.

Minimum size

`both quantities ≥ min_qty`

Token-sized quotes don't count as making a market.

`tick 0.01, max 10 ticks`

`BID 149.90 / ASK 150.10`

`spread = 20 ticks`

✗ **REJECTED**

`BID 149.95 / ASK 150.05`

`spread = 10 ticks`

✓ **ACCEPTED**

Quote lifecycle

- **ACTIVE** — both legs resting
- **ASK FILLED** — a customer bought
- **RE-QUOTE** — refresh with new size
- **CANCELLED** — replaced or pulled

Automated liquidity for a lively classroom

`pm-mm-bot`

The autonomous market maker

- **Quotes both sides** around the mid with a configurable gap; one bot per symbol.
- **Reprices on drift** when the mid moves too far, or when a side is filled.
- **Session-aware** — quotes only in CONTINUOUS, waits out auctions and halts.
- **Self-healing** — reconciles its legs if a message is dropped.

`pm-ai-trader / pm-ai-swarm`

Autonomous order flow

- **LIMIT-only bots** that always add depth — never sweep the book away.
- **A swarm of N** spawns many traders across symbols in one command.
- **Personality profiles** — aggressive, cautious, many-small, few-large.
- **Built-in guardrails** — position limits and a reject breaker per bot.

Day 3 in review — market structure

1. **Five session phases**
   Only CONTINUOUS matches; the rest gather or close.

2. **Phase gating**
   MARKET/FOK/IOC need CONTINUOUS; ATO/ATC live in auctions.

3. **Auctions cross at one price**
   Gather orders, then uncross at the equilibrium.

4. **Equilibrium = max volume**
   Ties break to smaller surplus, then lower price.

5. **Market makers quote two-sided**
   QUOTE posts both legs; obligations keep them honest.

6. **Bots supply liquidity**
   MM bots quote; AI swarms add realistic LIMIT flow.

KEY TAKEAWAY Structure turns a pile of orders into an orderly market — phases, auctions and standing liquidity.

DAY

04

DAY FOUR

Risk, Safety & Post-Trade

BY THE END OF TODAY YOU WILL BE ABLE TO

- Explain how price collars reject fat-finger orders before they rest
- Trace a circuit breaker from trigger to halt to auto-resume
- Compute realized and unrealized P&L as positions change
- Describe how audit and drop-copy give a complete compliance record

Price collars: the pre-trade safety net

Collars reject priced orders that sit too far from a reference — stopping fat-finger mistakes before they ever touch the book. Two bands guard every order.

STATIC BAND

± 20% from the opening reference

A wide, all-day safety net anchored at the day's reference price.

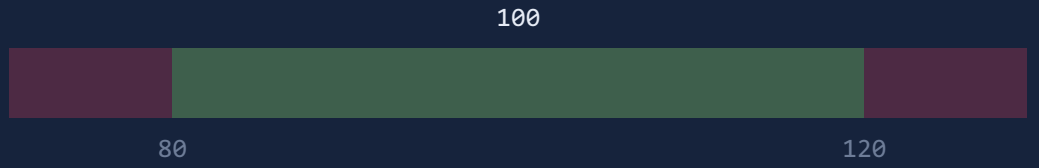
DYNAMIC BAND

± 2% from the last traded price

A tighter, rolling band — skipped until the first trade prints.

Worked example

Opening reference 100.00 → static band [80.00, 120.00]



✗ **SELL @ 75.00** below 80 → STATIC_COLLAR_BREACH

✓ **order @ 101.00** inside both bands → accepted

Circuit breakers & the kill switch

Where collars stop a bad order, a circuit breaker halts a whole symbol when executed trades move too far, too fast. The highest crossed level fires.

L3	**±20%**	Halt for the rest of the day
L2	**±13%**	Halt ~15 minutes
L1	**±7%**	Halt ~5 minutes

```
price_shift = | trade − reference | / reference
```

During a halt

- **MARKET / FOK / IOC** rejected; quotes cancelled.
- **LIMIT & ICEBERG rest** but do not match.
- **Auto-resume** runs an uncross, then continuous trading.

Kill switch — a scalpel, not a halt

Cancels one gateway's resting orders and quotes instantly — without halting the symbol and without needing an admin role. The trader-side emergency stop.

Lab: Trip a circuit breaker

EXERCISE

Your goal

Push a symbol until it breaches a breaker level, watch it halt, and confirm which order types survive the halt.

YOU SHOULD SEE
A halt event with trigger, reference and level — then auto-resume.

Step by step

1. **Seed a symbol at reference** `10000` with L1 = 7%
2. **Send rising trades** `10100` · `10700` · `11000` and watch the rolling reference climb
3. **See L1 fire near ~7% and the** `circuit_breaker.halt` event publish
4. **During the halt, try a MARKET (rejected) then a LIMIT** (accepted, but not matched).

P&L & clearing: tracking what you own

pm-clearing listens to every trade and maintains a position per gateway and symbol — net quantity, average cost, and both realized and unrealized P&L.

NET QTY

`+ LONG · − SHORT · 0 FLAT`

Your current exposure in the symbol.

AVG COST

`VOLUME-WEIGHTED ENTRY`

The average price you built the position at.

REALIZED P&L

`LOCKED IN ON CLOSE`

Profit banked when you reduce or close a position.

UNREALIZED P&L

`MARKED TO MARKET`

= net_qty × (mark − avg_cost), moves with the tape.

Four things a fill can do

- **OPEN** from flat
- **ADD** same side, re-average
- **REDUCE / CLOSE** realize P&L
- **CROSS ZERO** close then re-open the other side

Watch a position evolve

STEP	FILL	OUTCOME	NET	AVG	REALIZED
1	BUY 10 @ 100	Open long	+10	100.00	0
2	BUY 10 @ 110	Add · re-average	+20	105.00	0
3	SELL 20 @ 115	Close · (115−105)×20	0	—	+200
4	SELL 15 @ 108	Open short	−15	108.00	+200
5	BUY 20 @ 105	Cross zero · close 15	+5	105.00	+245

Cross-zero rule: only the closed portion realizes P&L; the excess opens the new side at the fill price with zero realized.

The compliance record: audit & drop-copy

`pm-audit`

Record everything, forever

- **Subscribes to every topic** and writes each event as one JSON line.
- **Durable & rotating** — timestamped logs that roll over and compress.
- **Full lifecycle** — orders, acks, fills, cancels, sessions, admin, breakers.
- **Query after the fact** — reconstruct any order's complete history.

`drop-copy :5557`

A live feed for risk & back-office

- **A parallel fill stream** on its own socket for compliance systems.
- **Two events per trade** — one for each counterparty.
- **Sequence-numbered** — a gap means a dropped message; replay to recover.
- **Complementary** — per-fill & replayable vs. the durable, every-topic log.

Day 4 in review — risk & post-trade

1. **Collars stop bad orders**
   Static ±20% and dynamic ±2% bands reject fat-fingers.

2. **Breakers halt symbols**
   Trades moving too far trip L1/L2/L3; highest level wins.

3. **Kill switch is surgical**
   Pulls one gateway's orders without halting the market.

4. **Positions track exposure**
   Net qty, average cost, realized & unrealized P&L.

5. **Cross-zero realizes carefully**
   Only the closed portion books P&L; the rest re-opens.

6. **Audit + drop-copy**
   A durable log plus a live, sequenced compliance feed.

KEY TAKEAWAY Safety and record-keeping are what separate a toy matcher from a real, trustworthy exchange.

DAY

05

DAY FIVE

Data, Monitoring & Operations

BY THE END OF TODAY YOU WILL BE ABLE TO

- Match each protocol — ALF, BALF, CALF, RALF, API — to its purpose
- Read OHLCV, VWAP and mid-price statistics and the market index
- Author, verify and launch a configuration the right way
- Bring every process together and run a full classroom exchange

Five ways in and out

EduMatcher speaks several protocols, each a separate gateway process with its own job. Choose by purpose — order entry, market data, post-trade or web.

ALF

Almost-FIX

Human-readable order entry — the trader terminal & TCP gateway.

BALF

Binary ALF

Low-latency binary order entry for programmatic clients.

CALF

Channel ALF

External market-data feed: subscribe to book, trades & index.

RALF

Reconciliation ALF

Post-trade dissemination to clearing, drop-copy & audit.

API

REST / WebSocket

HTTP + JSON front end for browsers, dashboards & simple bots.

One order can enter over ALF, print to CALF market-data subscribers, and reconcile over RALF — all at once.

Statistics: turning trades into insight

pm-stats subscribes to the tape and records it to a database; pm-stats-cli queries it — OHLCV bars, VWAP and periodic mid-price snapshots.

O	H	L	C	V	VWAP
150.00	153.25	149.50	152.75	5,000	151.82
					volume-weighted avg price

VWAP	Σ(price × qty) ÷ Σ(qty) — the average price the market actually paid.
MID PRICE	(best bid + best ask) ÷ 2, snapshotted on a timer as the book moves.
OHLCV	Open, high, low, close & volume — the daily bar for every symbol.

The market index in real time

pm-index watches trades and recomputes a market-cap-weighted index live — the same idea behind the S&P 500, kept continuous through corporate actions.

How the level is built

```
aggregate_cap = Σ(price × shares)
index_level   = aggregate_cap / divisor
```

- **The divisor is set** so the first level equals the base value.
- **Corporate actions rescale it** — splits & dividends keep the level continuous, not jumpy.

EDU100 — today

1,048.73

▲ +0.64% close · session CLOSED

Open	1,042.10
High	1,056.30
Low	1,040.05
Close	1,048.73

Lab: Read the tape

EXERCISE

Your goal

Pull the day's statistics from the recorder, hand-check a VWAP, and watch the index level track the session.

YOU SHOULD SEE
Your hand VWAP matching the recorder, to the cent.

Step by step

1. **Export a symbol's trades:** `pm-stats-cli --format csv trades --symbol AAPL`
2. **Hand-compute VWAP** as Σ(price×qty) ÷ Σ(qty) and compare with the stored value.
3. **Pull the daily bar:** `pm-stats-cli daily --symbol AAPL` check O/H/L/C/V
4. **Chart** `index-snapshots --index-id EDU100` across the session phases.

Operate safely: author, verify, then run

01 AUTHOR

`engine_config.yaml`

Symbols, gateways, sessions, risk & seeds — by CLI, GUI or by hand.

02 VERIFY

`pm-cverifier`

A read-only check that reports every error at once, plus a plain-English risk summary.

03 RUN

`pm-engine --verbose`

Binds the sockets and starts matching; other processes connect after it.

Verify before you run. The verifier surfaces undefined risk levels, port collisions and out-of-order schedules all at once — so you fix them on the ground, not mid-session. Exit codes make it CI-friendly.

The operator's console

From an ADMIN gateway you drive sessions and handle incidents. The same commands work interactively or scripted for automation.

SESSION & STATUS

SESSION_STATUS
read the current phase

SESSION|STATE=…
advance the phase

SCHEDULE
show auto-transitions

GATEWAYS
list connections

HALT & RESUME

HALT / RESUME
exchange-wide breaker

HALT_SYM|SYM=X
halt one symbol

RESUME_SYM|SYM=X
lift one symbol

BOOK|SYM=X
inspect a book

EMERGENCY

KILL|GW=X
cancel a gateway's orders

KICK|GW=X
disconnect a gateway

CANCEL_SYM|SYM=X
clear a whole symbol

QCANCEL|GW=MM
pull a quote

Capstone: the whole exchange, wired

Everything from the week, running at once. Start the engine first; data and gateway processes connect to its 5556 feed and fan the market out.

Day 5 in review — data & operations

1. **Protocols by purpose**
   ALF & BALF in, CALF data out, RALF post-trade, API for web.

2. **Statistics tell the story**
   OHLCV bars, VWAP and mid-price from the recorded tape.

3. **A live, continuous index**
   Cap-weighted, kept smooth through corporate actions.

4. **Author → verify → run**
   Never launch a config you haven't verified.

5. **The operator's console**
   Drive sessions, halt, resume, kill and kick from ADMIN.

6. **It all connects**
   Start the engine first; every process plugs into its feed.

KEY TAKEAWAY An exchange is only as good as the data it emits and the discipline with which it's run.

What you can do now

Five days ago, an exchange was a black box. Now you can reason about — and operate — every layer of one.

Read a live order book

and predict fills by price-time priority

Speak the language of orders

from MARKET to multi-leg combos

Reason about market structure

phases, auctions and the equilibrium price

Make markets & add liquidity

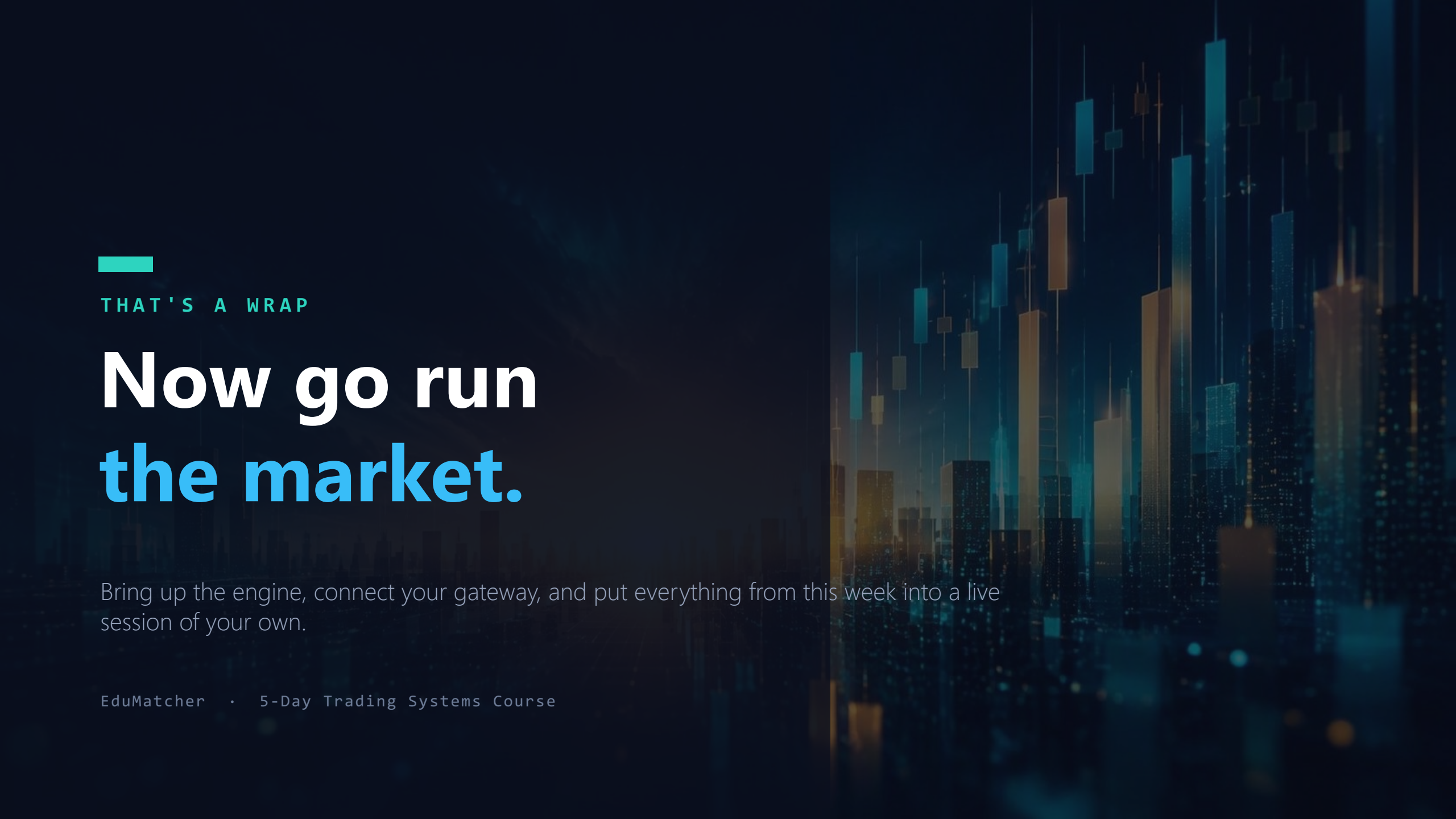
two-sided quotes, obligations, bots

Manage risk end-to-end

collars, breakers, kill switch, P&L

Operate a real exchange

configure, verify, monitor and run it



THAT'S A WRAP

Now go run the market.

Bring up the engine, connect your gateway, and put everything from this week into a live session of your own.

EduMatcher · 5-Day Trading Systems Course